МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
КАФЕДРА ИНТЕЛЛЕКТУАЛЬНЫХ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ

ОТЧЁТ

по лабораторной работе №5

Выполнил:
студент группы ПО-12
Снарский П.П.
Проверила:
Кулик А.Д.

Брест 2026

Цель работы: приобрести практические навыки разработки API и баз данных

Вариант 12

Ход работы

Общее задание:

1. Реализовать базу данных из не менее 5 таблиц на заданную тематику. При реализации продумать типизацию полей и внешние ключи в таблицах;
2. Визуализировать разработанную БД с помощью схемы, на которой отображены все таблицы и связи между ними (пример, схема на рис. 1);
3. На языке Python с использованием SQLAlchemy реализовать подключение к БД;
4. Реализовать основные операции с данными (выборку, добавление, удаление, модификацию);
5. Для каждой реализованной операции с использованием FastAPI реализовать отдельный эндпойнт;

Задание 1.

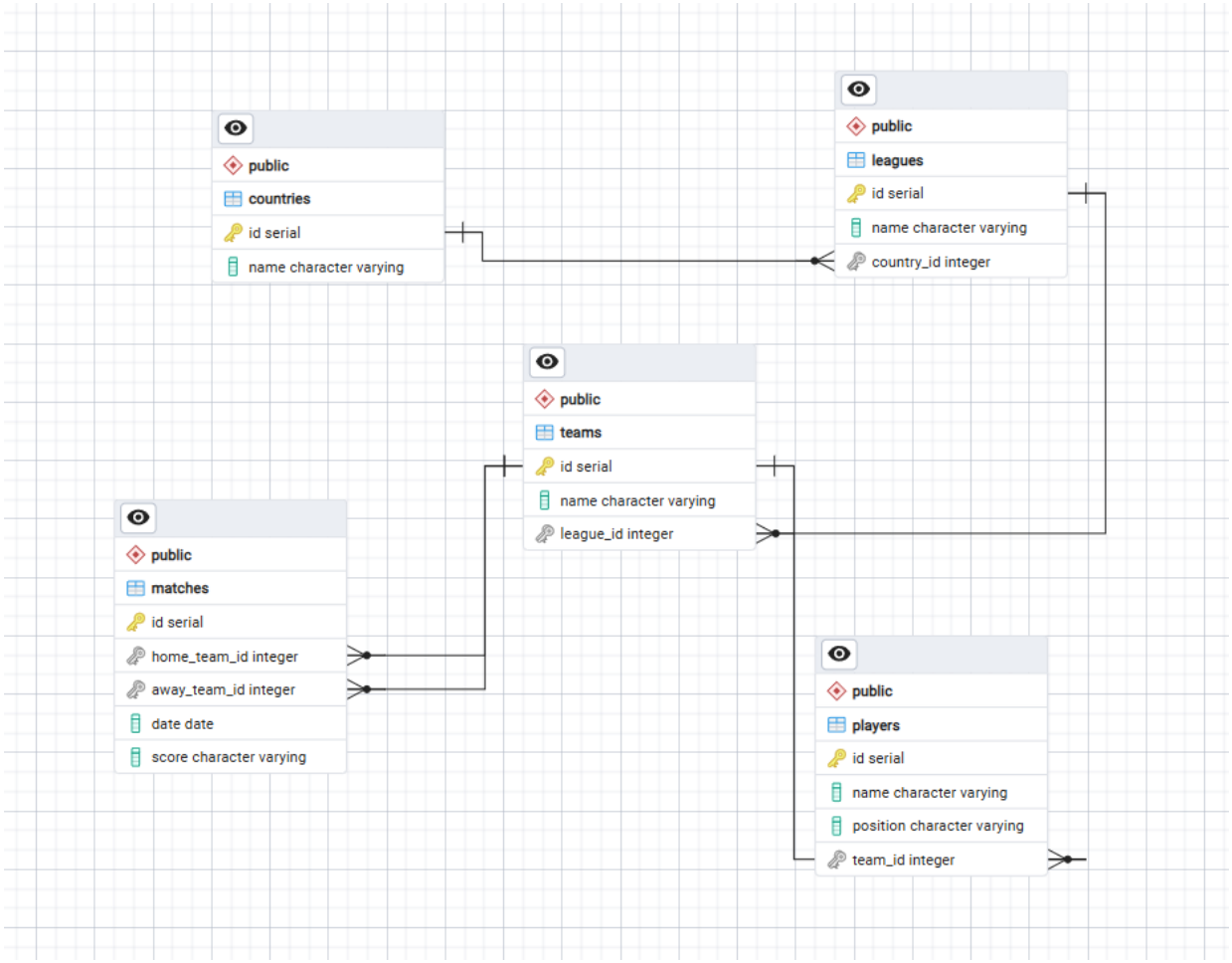
12) База данных Европейские футбольные чемпионаты

```
"""Database program"""

from sqlalchemy import Column, Integer, String, ForeignKey, Date,
create_engine
from sqlalchemy.orm import declarative_base, relationship, sessionmaker,
Session
from fastapi import FastAPI, Depends

Base = declarative_base()


class Country(Base):
    """Country class"""

    __tablename__ = "countries"

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)

    leagues = relationship("League", back_populates="country")


class League(Base):
    """League class"""

    __tablename__ = "leagues"

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    country_id = Column(Integer, ForeignKey("countries.id"))

    country = relationship("Country", back_populates="leagues")
    teams = relationship("Team", back_populates="league")


class Team(Base):
    """Team class"""
```

```python
    __tablename__ = "teams"

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    league_id = Column(Integer, ForeignKey("leagues.id"))

    league = relationship("League", back_populates="teams")
    players = relationship("Player", back_populates="team")


class Player(Base):
    """Player class"""

    __tablename__ = "players"

    id = Column(Integer, primary_key=True)
    name = Column(String, nullable=False)
    position = Column(String)
    team_id = Column(Integer, ForeignKey("teams.id"))

    team = relationship("Team", back_populates="players")


class Match(Base):
    """Match class"""

    __tablename__ = "matches"

    id = Column(Integer, primary_key=True)
    home_team_id = Column(Integer, ForeignKey("teams.id"))
    away_team_id = Column(Integer, ForeignKey("teams.id"))
    date = Column(Date)
    score = Column(String)


DATABASE_URL = "postgresql://postgres:1@localhost/football_db"

engine = create_engine(DATABASE_URL)
SESSION_LOCAL = sessionmaker(bind=engine)

Base.metadata.create_all(bind=engine)

app = FastAPI()


def get_db():
    """Get db fuction"""
    db = SESSION_LOCAL()
    try:
        yield db
    finally:
        db.close()


@app.post("/countries/")
def create_country(name: str, db: Session = Depends(get_db)):
    """Create country function"""
    country = Country(name=name)
    db.add(country)
    db.commit()
    db.refresh(country)
    return country
```

```python
@app.get("/countries/")
def get_countries(db: Session = Depends(get_db)):
    """Get countries function"""
    return db.query(Country).all()


@app.put("/countries/{country_id}")
def update_country(country_id: int, name: str, db: Session =
Depends(get_db)):
    """Update country function"""
    country = db.query(Country).get(country_id)
    country.name = name
    db.commit()
    return country


@app.delete("/countries/{country_id}")
def delete_country(country_id: int, db: Session = Depends(get_db)):
    """Delete country function"""
    country = db.query(Country).get(country_id)
    db.delete(country)
    db.commit()
    return {"message": "Deleted"}


@app.post("/teams/")
def create_team(name: str, league_id: int, db: Session = Depends(get_db)):
    """Create team function"""
    team = Team(name=name, league_id=league_id)
    db.add(team)
    db.commit()
    db.refresh(team)
    return team


@app.get("/teams/")
def get_teams(db: Session = Depends(get_db)):
    """Get teams function"""
    return db.query(Team).all()


@app.post("/players/")
def create_player(
    name: str, position: str, team_id: int, db: Session = Depends(get_db)
):
    """Create player function"""
    player = Player(name=name, position=position, team_id=team_id)
    db.add(player)
    db.commit()
    db.refresh(player)
    return player


@app.get("/players/")
def get_players(db: Session = Depends(get_db)):
    """Get players function"""
    return db.query(Player).all()


@app.post("/leagues/")
def create_league(name: str, country_id: int, db: Session = Depends(get_db)):
    """Create league function"""
    league = League(name=name, country_id=country_id)
    db.add(league)
```

```
    db.commit()
    db.refresh(league)
    return league


@app.get("/leagues/")
def get_leagues(db: Session = Depends(get_db)):
    """Get leagues function"""
    return db.query(League).all()


@app.get("/leagues/{league_id}")
def get_league(league_id: int, db: Session = Depends(get_db)):
    """Get league by id function"""
    return db.query(League).get(league_id)


@app.post("/matches/")
def create_match(
    home_team_id: int,
    away_team_id: int,
    date,
    score: str,
    db: Session = Depends(get_db),
):
    """Create match function"""
    match = Match(
        home_team_id=home_team_id, away_team_id=away_team_id, date=date,
score=score
    )
    db.add(match)
    db.commit()
    db.refresh(match)
    return match


@app.get("/matches/")
def get_matches(db: Session = Depends(get_db)):
    """Get matches function"""
    return db.query(Match).all()


@app.get("/matches/{match_id}")
def get_match(match_id: int, db: Session = Depends(get_db)):
    """Get match by id function"""
    return db.query(Match).get(match_id)
```



Вывод: в ходе выполнения лабораторной работы приобрел практические навыки разработки API и баз данных